

Sequence & Interaction Diagrams

Project: LearnLens AI

1. Flow 1 & 2: Start Learning & Chrome Tab Permission

```
sequenceDiagram
    autonumber
    actor Learner as Human Learner
    participant UI as LearnLens React Studio
    participant Browser as Chrome DisplayMedia API
    participant Server as FastAPI Backend Server
    participant DB as SQLite & File Storage

    Learner->>UI: Clicks "Start Learning"
    UI->>Learner: Displays Platform Selector Modal
    Learner->>UI: Submits Session Title & Platform
    UI->>Server: POST /api/sessions
    Server->>DB: Initialize session directory & DB record
    Server-->>UI: Returns LearningSession (ID)
    UI->>Browser: navigator.mediaDevices.getDisplayMedia({video: true, audio: true})
    Browser->>Learner: Native Chrome Picker (Tab / Window / Screen)
    Learner->>Browser: Selects Educational Tab + Checks "Share Audio"
    Browser-->>UI: Active MediaStream Granted
    UI->>Server: Connects ws://localhost:8000/ws/session/{id}
    UI->>Learner: Displays Live Capture Studio & Status Dots
```

2. Flow 3: Audio Transcription & Timestamp Sync

```
sequenceDiagram
    autonumber
    participant Client as Web Audio / Streamer
    participant WS as WebSocket Endpoint
    participant Whisper as faster-whisper Provider
    participant Knowledge as Knowledge Extractor
    participant DB as Session Database

    Client->>WS: Sends audio chunk / speech text with timestampSec
    WS->>Whisper: Pass audio frame for inference
    Whisper-->>WS: Returns transcript segment (text, confidence)
    WS->>DB: Store TranscriptSegment
    WS->>Knowledge: extract_knowledge_from_chunk(text, timestamp)
    Knowledge->>DB: Insert new Concepts & Definitions
    WS-->>Client: Broadcast event 'transcript_received' + new_concepts
```

3. Flow 4 & 5: Important Screenshot Detection & RAG Update

```
sequenceDiagram
    autonumber
    participant Canvas as Client Frame Sampler
    participant Svc as Screenshot Intelligence Service
    participant CV as OpenCV dHash Filter
    participant OCR as OCR / VLM Providers
    participant Vector as Vector & Embedding Store
```

```
Canvas->>Svc: process_frame_data(image_bytes, timestampSec)
Svc->>CV: Calculate 64-bit dHash
CV->>Svc: Compute Hamming Distance to previous frame
alt Distance < Threshold (Stationary Slide)
Svc-->>Canvas: SKIPPED_DUPLICATE (Drop Frame)
else Distance >= Threshold (Significant Scene Change)
Svc->>Svc: Save high-res JPEG & 320x180 thumbnail
Svc->>OCR: extract_text() + analyze_image()
OCR-->>Svc: Extracted OCR text & Visual Category (DIAGRAM/SLIDE)
Svc->>Vector: Generate Subword Vector Embeddings
Vector->>Vector: Update Session-Scoped Inverted Index
Svc-->>Canvas: Returns FrameCapture (Broadcast to UI)
end
```

4. Flow 6 & 7: User Asks Doubt & AI Answers With Evidence

```
sequenceDiagram
autonumber
actor Learner as Human Learner
participant UI as Chat Interface
participant Teacher as AI Teacher Service
participant RAG as Session-Scoped RAG
participant LLM as Ollama / Local Heuristic LLM

Learner->>UI: "What did the instructor teach about firewalls?"
UI->>Teacher: POST /api/sessions/{id}/chat (message, mode='simple')
Teacher->>RAG: retrieve_context(sessionId, "firewalls", topK=4)
RAG->>RAG: Match concepts, transcripts, and frame OCR in session namespace
RAG-->>Teacher: Grounded Context + Evidence Cards (Timestamps & Thumbnails)
Teacher->>LLM: Prompt with Mode Template + Grounded Source Excerpts
LLM-->>Teacher: Synthesized Pedagogical Explanation
Teacher-->>UI: ChatMessage (text, citations, thumbnail URLs)
UI->>Learner: Renders response with clickable timestamp pills and thumbnail cards
Learner->>UI: Clicks thumbnail card
UI->>Learner: Opens High-Res Slide Inspection Modal
```

5. Flow 8: PowerPoint Generation (`python-pptx`)

```
sequenceDiagram
autonumber
actor Learner as Human Learner
participant UI as Study Artifacts View
participant PPT as PPT Generation Service
participant DB as SQLite DB & Screenshots
participant File as Filesystem (.pptx)

Learner->>UI: Clicks "Generate Presentation" (Style='Teaching', Count=8)
UI->>PPT: POST /api/sessions/{id}/generate-ppt
PPT->>DB: Fetch session concepts, verified frames, and transcript highlights
PPT->>PPT: Initialize 16:9 widescreen presentation layout
PPT->>PPT: Build Title Slide & Module Overview
PPT->>PPT: Build Concept Definition Cards
PPT->>DB: Retrieve top diagram screenshot file
PPT->>PPT: Embed actual captured diagram picture on visual slide
PPT->>PPT: Build Real-World Analogy & High-Yield Exam Traps slides
PPT->>File: Save presentation to exports/LearnLens_...pptx
PPT-->>UI: Returns { status: 'SUCCESS', download_url }
UI->>Learner: Displays "Download Presentation" button
```

6. Flow 9: PDF Study Pack Generation (`reportlab`)

```
sequenceDiagram
    autonumber
    actor Learner as Human Learner
    participant UI as Study Artifacts View
    participant PDF as PDF Generation Service
    participant File as Filesystem (.pdf)

    Learner->>UI: Clicks "Generate Teaching Report PDF"
    UI->>PDF: POST /api/sessions/{id}/generate-pdf (type='teaching_report')
    PDF->>PDF: Compile Executive Summary, Concept Table, Mechanisms, Exam Tips
    PDF->>File: Render PDF document via ReportLab SimpleDocTemplate
    PDF-->>UI: Returns { status: 'SUCCESS', download_url }
    UI->>Learner: Renders instant download link
```

7. Flow 10: Import Quiz PDF & Gap Diagnosis

```
sequenceDiagram
    autonumber
    actor Learner as Human Learner
    participant UI as Practice Quiz Modal
    participant QuizSvc as Quiz Preparation Service
    participant PyMuPDF as PyMuPDF Extractor
    participant LearnerProf as Learner Memory Model

    Learner->>UI: Uploads practice quiz PDF
    UI->>QuizSvc: POST /api/sessions/{id}/quiz/import (file)
    QuizSvc->>PyMuPDF: Extract text, options, answers, and explanations
    QuizSvc->>QuizSvc: Map questions to captured session concepts
    QuizSvc-->>UI: Returns parsed QuizQuestion objects
    Learner->>UI: Selects Option B on Question 1
    UI->>QuizSvc: POST /api/quiz/{qId}/submit (choice=1)
    QuizSvc->>QuizSvc: Verify correctness against grounded source rule
    QuizSvc->>LearnerProf: Update concepts_grasped or weak_areas
    QuizSvc-->>UI: Returns verification, explanation, and weak area alerts
    UI->>Learner: Displays green/red validation and "Teach Concept" button
```

8. Flow 11 & 12: Human Interrupts & Resumes Agent

```
sequenceDiagram
    autonumber
    actor Learner as Human Learner
    participant UI as Human Control Bar
    participant TaskSvc as Task Agent Service
    participant Capture as Browser Capture Stream

    Note over Learner,TaskSvc: Agent is actively processing lecture (OBSERVING)
    Learner->>UI: Clicks [ PAUSE AGENT ]
    UI->>TaskSvc: POST /api/sessions/{id}/pause
    TaskSvc->>TaskSvc: Transition state: PAUSED
    TaskSvc-->>UI: Status updated to PAUSED
    UI->>Learner: Visual banner: "Agent Paused. Ready for commands."
    Learner->>UI: Enters command: "Explain the previous slide"
    UI->>TaskSvc: POST /api/sessions/{id}/command
    TaskSvc->>UI: Returns explanation without resuming capture
    Learner->>UI: Clicks [ RESUME AGENT ]
    UI->>TaskSvc: POST /api/sessions/{id}/resume
```

```
TaskSvc->>TaskSvc: Transition state: OBSERVING
UI->>Capture: Resumes frame and audio ingestion
```

9. Flow 13: Delete Session & Purge Local Files

```
sequenceDiagram
    autonumber
    actor Learner as Human Learner
    participant UI as History Sidebar
    participant Server as FastAPI Server
    participant DB as SQLite DB
    participant Disk as Local Storage

    Learner->>UI: Clicks Trash icon on session
    UI->>Server: DELETE /api/sessions/{id}
    Server->>DB: Cascading delete: sessions, transcripts, frames, concepts, chats
    Server->>Disk: Remove directory data/sessions/{id}/ (screenshots + exports)
    Server-->>UI: { status: 'DELETED' }
    UI->>Learner: Removes session from UI and switches to next available
```